

# **Review of C-Programming: Bitwise Operation (for Embedded Controller)**

**Prof. Young-Keun Kim**

# Firmware programming example

- Example of firmware programming in C

#Define

Pointer

Typecasting

Enum

Structures

Bitwise

```
#ifndef __EC_GPIO_H
#define __EC_GPIO_H

#define INPUT 0x00
#define GPIOA ((GPIO_TypeDef *) GPIOA_BASE)

typedef enum
{
    /***** Cortex-M4 Processor Exceptions Numbers *** */
    RCC_IRQn = 5,    /*!< RCC global Interrupt */
    EXTI0_IRQn = 6,  /*!< EXTI Line0 Interrupt */
    EXTI1_IRQn = 7,  /*!< EXTI Line1 Interrupt */
}

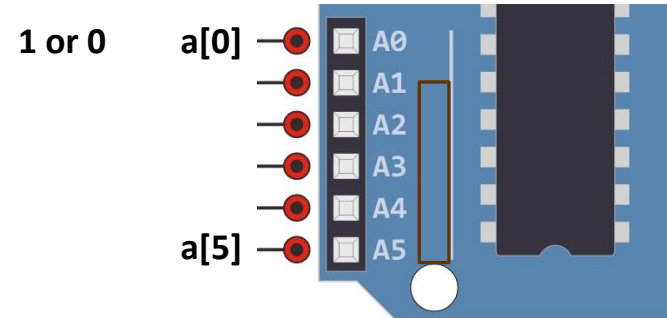
typedef struct
{
    __IO uint32_t MODER; /*!< GPIO port mode register, Address offset: 0x00 */
    __IO uint32_t OTYPER; /*!< GPIO port output type register, Address offset: 0x04 */
    __IO uint32_t ODR; /*!< GPIO port output data register, Address offset: 0x14 */
} GPIO_TypeDef;

// GPIO Mode : Input(00), Output(01), AlterFunc(10), Analog(11, reset)
void GPIO_mode(GPIO_TypeDef* Port, int pin, int mode) {
    Port->MODER &= ~(3UL << (2 * pin));
    Port->MODER |= mode << (2 * pin);
}
```

# Bitwise Operation

- [Tutorial: Bitwise Operation](#)

- Why use Bitwise operation?



`uint8_t a=5`

a has 8-bits memory (NOT 8-bytes)

|        |   |                |                |                |                |                |                |                |                |
|--------|---|----------------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|
| memory | a | a <sub>7</sub> | a <sub>6</sub> | a <sub>5</sub> | a <sub>4</sub> | a <sub>3</sub> | a <sub>2</sub> | a <sub>1</sub> | a <sub>0</sub> |
| value  | 5 | 0              | 0              | 0              | 0              | 0              | 1              | 0              | 1              |

↑

How to read or set only the bit at **a[5]**? // it is not 1D array index

- MCU I/O pin is binary (1 or 0).
- Each pin is connected to one bit (not bytes) → Need to access and modify bits
- e.g. Turn on only the LED on pin **a[5]**

# Shift Operator

## ● Shift

- For MCU, we only use logical shift (not arithmetic)
- Logical shift: Fill the blanks with '0', regardless of sign
  - $s \ll n$ : shift to left by n digits
  - $s \gg n$ : shift to right by n digits

**(1<<2), (1>>2)**

unsigned s = 0001 (1)

(LSL)  $S \ll 2 \rightarrow 0100$  (4)

(LSR)  $S \gg 2 \rightarrow 0000$  (0)

**(4<<2), (4>>2)**

unsigned s = 0 0100 (4)

(LSL)  $S \ll 2 \rightarrow 1\ 0000$  (16)

(LSR)  $S \gg 2 \rightarrow 0\ 0001$  (1)

- Multiply or divide by  $2^n$  for unsigned number

(VAL<<4); // Multiply VAL by  $2^4=16$

(VAL>>4); // Divide VAL by  $2^4=16$

unsigned s=0110 0100 (100)

(LSL)  $S \ll 2 \rightarrow 1001\ 0000$  (144) \*OVF

(LSR)  $S \gg 2 \rightarrow 0001\ 1001$  (25)

# Bit Operator vs Boolean Operator

- Need to know how to read and control individual bits of I/O memory
  - Check / Set / Reset / Toggle bit(s)
- Bit operators
  - Bit Operators:  $(\&, |, \sim)$
  - Boolean Operators:  $(\&\&, ||, !)$  // True or False

|        |             |       |             |
|--------|-------------|-------|-------------|
| A && B | Boolean AND | A & B | Bitwise AND |
| A   B  | Boolean OR  | A   B | Bitwise OR  |
| !B     | Boolean NOT | ~B    | Bitwise NOT |
|        |             | A ^ B | Bitwise XOR |

| Code        | Result | Comment |
|-------------|--------|---------|
| b10 & b01   | b00    | Bitwise |
| b10 && b01  | b01    | Boolean |
| ~(0x01)     |        | Bitwise |
| !(0x01)     |        | Boolean |
| 0x11 ^ 0x01 |        | Bitwise |

# Bit Read

- Read the bit at  $k^{\text{th}}$  index:  **$\text{bit} = (a \gg k) \& 1$**

|   |   |   |   |   |
|---|---|---|---|---|
| 1 | & | 1 | = | 1 |
| 0 | & | 1 | = | 0 |

Example: read bit at  $a[5]$  from 8-bit number  $a$

[Method 1]  
(recommend)

|                                          |       |       |                         |       |       |       |       |                         |
|------------------------------------------|-------|-------|-------------------------|-------|-------|-------|-------|-------------------------|
| <b>a</b>                                 | $a_7$ | $a_6$ | <b><math>a_5</math></b> | $a_4$ | $a_3$ | $a_2$ | $a_1$ | $a_0$                   |
| <b><math>a \gg 5</math></b>              | 0     | 0     | 0                       | 0     | 0     | $a_7$ | $a_6$ | <b><math>a_5</math></b> |
| <b><math>(a \gg 5) \&amp; (1)</math></b> | 0     | 0     | 0                       | 0     | 0     | 0     | 0     | <b><math>a_5</math></b> |

[Method 2]

|                                        |       |       |                         |       |       |       |       |       |
|----------------------------------------|-------|-------|-------------------------|-------|-------|-------|-------|-------|
| <b>a</b>                               | $a_7$ | $a_6$ | <b><math>a_5</math></b> | $a_4$ | $a_3$ | $a_2$ | $a_1$ | $a_0$ |
| <b><math>1 \ll 5</math></b>            | 0     | 0     | <b>1</b>                | 0     | 0     | 0     | 0     | 0     |
| <b><math>a \&amp; (1 \ll 5)</math></b> | 0     | 0     | <b><math>a_5</math></b> | 0     | 0     | 0     | 0     | 0     |

Exercise: Read  $a[6]$  from port  $a$

**$\text{uint8\_t bit} = a \_\_\_\_\_\_ \& \_\_\_\_\_\_; // \text{ check bit of } a_6$**

## Bit Read

- **Reading multiple bits:**

(warning) read value is not binary anymore. 2 bits --> (decimal 0~3)

Exercise: Read a[6] and a[5] (2-bits) from port **a**

|            |                |                |                |                |                |                |                |                |
|------------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|
| <b>a</b>   | a <sub>7</sub> | a <sub>6</sub> | a <sub>5</sub> | a <sub>4</sub> | a <sub>3</sub> | a <sub>2</sub> | a <sub>1</sub> | a <sub>0</sub> |
|            |                |                |                |                |                |                |                |                |
|            |                |                |                |                |                |                |                |                |
| <b>val</b> |                |                |                |                |                |                |                |                |

**uint8\_t val=\_\_\_\_\_ & \_\_\_\_\_; // check bits of a6, a5**

## Bit Read

- **Reading multiple bits:**

Example: read CKD[1:0] from TIM1\_CR1

### TIM1 and TIM8 control register 1 (TIMx\_CR1)

[illegible]

```
uint16_t ClkDiv= (TIM1_CR1>>8) & 0x03
```

```
uint16_t ClkDiv=
```

|          |    |    |    |    |    |   |   |          |   |   |   |   |   |   |   |          |    |
|----------|----|----|----|----|----|---|---|----------|---|---|---|---|---|---|---|----------|----|
| 15       | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7        | 6 | 5 | 4 | 3 | 2 | 1 | 0 |          |    |
| Reserved |    |    |    |    |    |   |   | Reserved |   |   |   |   |   |   |   | CKD[1:0] |    |
|          |    |    |    |    |    |   |   |          |   |   |   |   |   |   |   | rw       | rw |

# Bit Set

- Set(HIGH) a bit in a number

$$a = a \mid (1 \ll k) \quad \text{or} \quad a \mid= (1 \ll k)$$

|   |  |   |   |   |
|---|--|---|---|---|
| X |  | 1 | = | 1 |
| X |  | 0 | = | X |

← masking

Example: `a =a | (1<<5);     // set a5`

|                           |                |                |                      |                |                |                |                |                |
|---------------------------|----------------|----------------|----------------------|----------------|----------------|----------------|----------------|----------------|
| <b>a</b>                  | a <sub>7</sub> | a <sub>6</sub> | <b>a<sub>5</sub></b> | a <sub>4</sub> | a <sub>3</sub> | a <sub>2</sub> | a <sub>1</sub> | a <sub>0</sub> |
| <b>1 &lt;&lt; 5</b>       | 0              | 0              | <b>1</b>             | 0              | 0              | 0              | 0              | 0              |
| <b>a   (1 &lt;&lt; 5)</b> | a <sub>7</sub> | a <sub>6</sub> | <b>1</b>             | a <sub>4</sub> | a <sub>3</sub> | a <sub>2</sub> | a <sub>1</sub> | a <sub>0</sub> |

- Exercise: Turning ON LEDs of Port A(PA)
  - assume 8 LEDs are connected to Digital Out pins of PA

`PA=b00001111;`                      `//LED0 is LSB, Set to turn on LED`

`PA|=_____;`                      `// turn ON LED4`

`PA|=_____;`                      `// turn ON LED4 and LED5`

# Bit Clear

- Clear(LOW) a bit in a number

**$a \&= \sim(1 \ll k)$**

$X \& 0 = 0$   
 $X \& 1 = X$

masking

Example:  $a \&= \sim(1 \ll 5);$  // clear bit a5

$\sim(1 \ll 5) \rightarrow \sim(0010\ 0000) \rightarrow (1101\ 1111)$

|                      |                |                |                |                |                |                |                |                |
|----------------------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|
| a                    | a <sub>7</sub> | a <sub>6</sub> | a <sub>5</sub> | a <sub>4</sub> | a <sub>3</sub> | a <sub>2</sub> | a <sub>1</sub> | a <sub>0</sub> |
| $\sim(1 \ll 5)$      | 1              | 1              | 0              | 1              | 1              | 1              | 1              | 1              |
| $a \& \sim(1 \ll 5)$ | a <sub>7</sub> | a <sub>6</sub> | 0              | a <sub>4</sub> | a <sub>3</sub> | a <sub>2</sub> | a <sub>1</sub> | a <sub>0</sub> |

- Exercise: Turning off LEDs of Port A(PA)

PA=b00001111; //LED0 is LSB, Set to turn on LED

**PA&=\_\_\_\_\_;** // turn off LED2

**PA&=\_\_\_\_\_;** // turn off LED0 and LED2

# Bit Clear

- Toggle a bit

$$a \wedge = 1 \ll k$$

$$\begin{aligned} X \wedge 1 &= \sim X && \leftarrow \text{(XOR)} \\ X \wedge 0 &= X && \leftarrow \text{masking} \end{aligned}$$

Without knowing the initial value, a bit can be toggled by XORing it with a “1”

**Example:** Blink LED5 on and off,  $k = 5$ .

|                      |       |       |              |       |       |       |       |       |
|----------------------|-------|-------|--------------|-------|-------|-------|-------|-------|
| $a$                  | $a_7$ | $a_6$ | $a_5$        | $a_4$ | $a_3$ | $a_2$ | $a_1$ | $a_0$ |
| $1 \ll k$            | 0     | 0     | 1            | 0     | 0     | 0     | 0     | 0     |
| $a \wedge = 1 \ll k$ | $a_7$ | $a_6$ | NOT( $a_5$ ) | $a_4$ | $a_3$ | $a_2$ | $a_1$ | $a_0$ |

| $a_5$ | 1 | $a_5 \oplus 1$ |
|-------|---|----------------|
| 0     | 1 | 1              |
| 1     | 1 | 0              |

# Bit Replace

- Replacing multiple bits

TIP: Always (1) First Reset then (2) Write

Example: replace oldMin by newMin at time[5-9] // 5 bits

|             |              |            |            |
|-------------|--------------|------------|------------|
| bits 15 -10 | bits 9 - 5   | bits 4 - 0 |            |
| Hours       | OldMin       | Seconds    | time (old) |
| Hours       | <b>00000</b> | Seconds    |            |
| Hours       | newMin       | Seconds    | time (new) |

  
b11111 (0x1F)

$\text{time} = \text{time} \& \sim(0x1F \ll 5)$  // First, reset bit (5~9 index)

$\text{time} | = (\text{newMin} \& 0x1F) \ll 5$  // Then, write bits

## Bitwise Operation: Summary

- Set(HIGH) a bit in a number

**$a \mid= (1 \ll k)$**

$$\begin{array}{l} X \ \& \ 0 = 0 \\ X \ \& \ 1 = X \end{array}$$

- Clear(LOW) a bit in a number

**$a \ \&= \sim(1 \ll k)$**

$$\begin{array}{l} X \ \mid \ 1 = 1 \\ X \ \mid \ 0 = X \end{array}$$

- Read/Check a bit:

**$\text{bit} = (a \gg k) \ \& \ 1$**

$$\begin{array}{l} 1 \ \& \ 1 = 1 \\ 0 \ \& \ 1 = 0 \end{array}$$

- Toggle a bit

**$a \ \wedge= 1 \ll k$**

$$X \wedge 1 = \sim X \quad \leftarrow \text{(XOR)}$$

// Macro defined in stm32fxx.h

```
#define SET_BIT(REG, BIT)      ((REG) |= (BIT))
#define CLEAR_BIT(REG, BIT)   ((REG) &= ~(BIT))
#define READ_BIT(REG, BIT)    ((REG) & (BIT))
```

# Bitwise Operation: Exercise

---

## Exercise: Bitwise Operation